

Projet de fin de formation —

Spécialisation Data Scientist

Scoring de churn client — Telco Customer Churn (CSV)

Version apprenant — Décembre 2025

Contexte

Vous endossez le rôle de Data Scientist au sein de **TelcoWave**, un opérateur télécom (mobile + fibre) présent en Europe. La direction « Customer Success » vous confie un enjeu prioritaire : **réduire le churn** (résiliations) au prochain trimestre.

L'entreprise souhaite mettre en place un programme de rétention ciblé (appels sortants, remise commerciale, changement d'offre). Votre mission : **construire un modèle de scoring** capable d'estimer la probabilité de churn pour chaque client, afin de prioriser les actions sur les clients les plus à risque, avec un budget marketing limité.

Le livrable attendu est un dépôt Git reproductible contenant l'analyse exploratoire, un modèle baseline, puis un modèle finetuné accompagné d'une recommandation de seuil/stratégie de ciblage (top K% ou seuil de probabilité).

Objectifs pédagogiques

À l'issue du projet, vous aurez (1) structuré un dépôt Git professionnel, (2) réalisé une EDA complète en Python, (3) construit une **pipeline de préparation des données** (nettoyage, encodage) et livré un modèle baseline, puis (4) amélioré la performance via un **finetuning** (modèle, features, hyperparamètres, calibration, choix de seuil). La rigueur méthodologique, la reproductibilité et la capacité à relier la métrique ML à une décision métier seront particulièrement évaluées.

Données et cible

Le jeu de données fourni est un extrait anonymisé de TelcoWave (un enregistrement par client). Il contient des informations **démographiques**, des **services souscrits**, des **informations contractuelles** et de la **facturation** (tenure, MonthlyCharges, TotalCharges).

Variable cible : Churn (Yes/No) — le client a résilié dans la dernière période observée.

Format : **CSV fourni par l'équipe pédagogique** (issu du dataset public Kaggle « Telco Customer Churn »).

Étape 1 — Création du repo Git + EDA

Objectif : mettre en place un dépôt propre et reproductible, puis **comprendre le dataset** (qualité, distributions, relations avec la cible). Cette étape se fait exclusivement en Python (notebook).

Travail attendu :

- 1 Initialiser le dépôt Git (README, .gitignore, structure de dossiers) et réaliser au moins 3 commits propres (setup, EDA, baseline plan).
- 2 Charger le CSV et produire un mini dictionnaire de données (définition, type, exemple).
- 3 Contrôler la qualité : valeurs manquantes (dont TotalCharges souvent au format texte), doublons, valeurs aberrantes, cohérences (tenure ≥ 0 , charges ≥ 0).
- 4 Explorer la cible : taux de churn global et par segments (contrat, internet service, tenure, payment method, etc.).
- 5 Définir votre protocole d'évaluation : split train/valid/test (ou CV), métriques retenues, gestion du déséquilibre éventuel.

Étape 2 — Livrer le modèle Baseline (préparation + encodage)

Objectif : livrer un premier modèle opérationnel de scoring, avec une pipeline de préparation des données complète. Le modèle baseline doit être simple, robuste et reproductible (scikit-learn recommandé).

Attendus (techniques) :

- 1 Créer une pipeline de préparation : séparation X/y, split, traitement des valeurs manquantes, typage, encodage des catégories (One-Hot), scaling des numériques si besoin.
- 2 Mettre en place une **ColumnTransformer** et une **Pipeline** scikit-learn pour éviter les fuites de données.
- 3 Entraîner un modèle baseline (ex. LogisticRegression, DummyClassifier comme point de référence, ou arbre simple).
- 4 Évaluer le modèle : ROC-AUC (minimum), matrice de confusion, précision/rappel/F1, et au moins une métrique orientée « top ciblage » (ex. recall@10% ou precision@10%).
- 5 Sauvegarder le modèle (joblib/pickle) et produire un fichier de scoring (ex. customerID, proba_churn) sur un jeu de test.

Livrable intermédiaire (Checkpoint 2) : modèle baseline + rapport de métriques + fichier de scoring.

Étape 3 — Livrer le modèle Finetuné (optimisation)

Objectif : améliorer la performance et la pertinence métier du scoring, en documentant précisément les gains et les compromis. Vous devez comparer votre modèle finetuné au baseline avec un protocole d'évaluation identique.

Pistes d'amélioration possibles (à sélectionner et justifier) :

- 1 **Modèle** : essayer un modèle non-linéaire (RandomForest, GradientBoosting, HistGradientBoosting, XGBoost/LightGBM si autorisé).
- 2 **Feature engineering** : transformations (log de charges), regroupement de modalités rares, indicateurs (ex. a_totalcharges_is_missing).
- 3 **Hyperparamètres** : RandomizedSearchCV / GridSearchCV + validation croisée, en contrôlant le temps de calcul.
- 4 **Calibration** : vérifier la calibration des probabilités (courbe de calibration, Brier score) et calibrer si utile.
- 5 **Choix de seuil** : optimiser un seuil selon un coût/bénéfice (ex. coût offre = 15€, valeur sauvée = 120€) ou choisir un top K%.
- 6 **Interprétabilité** : importance des variables (permutation importance), analyse des segments à risque.

Livrable intermédiaire (Checkpoint 3) : modèle finetuné + rapport comparatif + scoring final.

Livrables et structure du dépôt Git

- 1 **Dépôt Git** complet (README, .gitignore, historique de commits lisible).
- 2 **Notebooks** : EDA (étape 1), baseline (étape 2), finetune (étape 3).
- 3 **Artefacts modèles** : baseline et finetuné (joblib), dossier models/.
- 4 **Scoring** : CSV final (customerID, proba_churn, label_pred) + rapport (Markdown/PDF).

Organisation recommandée du dépôt :

```
/
|-- README.md
|-- data/
|   |-- raw/
|   |-- telco_customer_churn.csv
|-- notebooks/
|   |-- 01_setup_repo_et_eda.ipynb
|   |-- 02_baseline_model.ipynb
|   |-- 03_finetuned_model.ipynb
|-- src/
|   |-- data_prep.py
|   |-- train.py
|   |-- infer.py
|   |-- metrics.py
|-- models/
|   |-- baseline.joblib
|   |-- finetuned.joblib
|-- reports/
|   |-- figures/
|   |-- model_report.md
|-- requirements.txt
-- .gitignore
```

README — contenu minimal :

- 1) Contexte métier et objectif du scoring.
- 2) Description des données (colonnes, cible, qualité).
- 3) Installation (environnement, dépendances).
- 4) Reproduire : EDA → baseline → finetune → scoring.
- 5) Résultats (métriques) et décision de seuil / top K%.
- 6) Limites/risques (fuite de données, biais) et pistes d'amélioration.

Grille d'évaluation (synthèse)

- 1 **Cadrage & traçabilité** : repo propre, structure cohérente, hypothèses explicites, reproductibilité.
- 2 **EDA** : qualité des contrôles, analyses pertinentes, visualisations lisibles, compréhension de la cible.
- 3 **Pipeline ML** : preprocessing correct (pas de leakage), encodage/typage, split/validation rigoureux.
- 4 **Baseline** : point de comparaison clair (Dummy + modèle simple), métriques adaptées et commentées.
- 5 **Finetuning** : amélioration démontrée, hyperparamètres/features justifiés, calibration/seuil traités.
- 6 **Interprétabilité & recommandation** : facteurs clés, segments à risque, ciblage actionnable.
- 7 **Qualité de code** : lisibilité, factorisation (src/), seeds, commentaires utiles.

Conseils pratiques

- 1 Commencez par un **DummyClassifier** puis un baseline interprétable (logistic regression).
- 2 Assurez-vous que **toutes les transformations** apprennent uniquement sur le train (Pipeline).
- 3 Contrôlez **TotalCharges** : il peut contenir des valeurs vides ou être lu comme texte.
- 4 Changez une chose à la fois, et documentez l'impact (expérimentation).
- 5 Pensez « décision » : à quel coût, pour combien de clients, et quel gain attendu ?

Note : le dataset public Kaggle s'appelle souvent *WA_Fn-UseC_-Telco-Customer-Churn.csv*. Vous pouvez le renommer en *telco_customer_churn.csv* dans *data/raw/* pour standardiser le projet.